

# Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk

HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

## Thema 5 - Übersicht

AV-Pipeline und Synchronisation - Mediendateien prüfen, zerlegen, synchronisieren und wieder ausgeben

- Container und Streams
- Muxen und Demuxen
- Zeitbezug und Synchronisation

## Thema 5 - Lernziel

Die Studierenden analysieren eine AV-Datei als Container mit mehreren Streams, setzen einfache Schritte zum Inspektieren, Demuxen, Muxen und Diagnostizieren von Synchronisation mit Python um und begründen, welcher Schritt in welchem Praxisfall sinnvoll ist.

## AV-Pipeline und Synchronisation: Mediendateien analysieren und Bearbeitung vorbereiten

Thema 5: Dateien analysieren, Bearbeitung vorbereiten

- Container und Streams
- Demuxen/Muxen
- Framerate, Samplerate, Zeitbezug
- Timestamps, Latenz, AV-Synchronisation
- Remuxing, Transcoding, Schneiden, Konvertieren

nächster Termin: Thema 6: verarbeiten, verändern, korrigieren

- Audio-Video-Editing
- Synchronisations-Korrekturen

## AV-Pipeline und Synchronisation: Mediendateien analysieren und Bearbeitung vorbereiten

- reale Mediendatei als Arbeitsmaterial
- Praxisfall:
  - Smartphone-Interviewclip
  - schlechter Kameraton
  - externer Audiotrack
  - spätere Weiterverarbeitung in Python / OpenCV
- Ziel:
  - Zusammensetzung des Videos verstehen
  - Video analysieren
  - Ersatz vorbereiten

## Struktur der Datei

- vor der Weiterverarbeitung:
  - welche Streams liegen vor?
  - welche Parameter sind relevant?
  - wie extrahiert man Audio und Video getrennt?
- bei der Nutzung:
  - Audio extrahieren
  - Video analysieren
  - externen Ton implementieren
  - Sync-Probleme erkennen

## Pipeline

- Kette aufeinander bezogener Verarbeitungsschritte
- in AV:
  - Quelle bzw. Aufnahme
  - Container/Streams
  - Demuxing
  - ggf. Decoding
  - Analyse, Bearbeitung
  - ggf. Encoding
  - Muxing
  - Ausgabe
- Tools: FFmpeg, OpenCV, Python.

An welcher Stelle der Pipeline würden Sie OpenCV verorten – vor oder nach dem Decoding?  
Begründung?

OpenCV sitzt in der AV-Pipeline typischerweise hinter Demuxing und Decoding, weil es mit decodierten Frames und nicht mit komprimierten Videopaketen arbeitet.

## Container und Streams

Container: z.B. MP4, MOV, MKV

im Container:

- Video
- Audio
- Untertitel

Werkzeug `ffprobe`:

- `-show_format`
- `-show_streams`
- JSON-Ausgabe

## Container und Streams

- professionelle Film- und TV-Workflows:
  - Audioanteile liegen oft getrennt als Stems vor, z. B. Dialog, Musik, Effekte, Atmo
- Für internationale Fassungen: dialogfreie M&E-/IT-Spur wichtig:  
Music & Effects ohne Originaldialog.
- Endnutzer-File: Bestandteile meist nicht mehr getrennt, sondern zu fertigen Mischungen zusammengeführt

## Beispiel 16: Videodatei analysieren

- Python + `ffprobe`
- JSON parsen
- Container, Dauer, Audio- und Videostreams ausgeben
- erste Arbeitsentscheidung
  
- `-show_format` → Informationen zum Container
- `-show_streams` → Informationen zu den einzelnen Streams
- `-of json` → strukturierte Ausgabe für Python

## Beispiel 16: Videodatei analysieren

```
...
for s in info["streams"]:
    typ = s["codec_type"]
    codec = s.get("codec_name", "unbekannt")
    print(f"Stream {s['index']}: {typ}, Codec={codec}")
    if typ == "video":
        video_found = True
        print("  Auflösung:", s.get("width"), "x", s.get("height"))
        print("  FPS:", s.get("r_frame_rate"))
    if typ == "audio":
        audio_found = True
        print("  Samplerate:", s.get("sample_rate"))
        print("  Kanäle:", s.get("channels"))
if video_found and audio_found:
    print("Praxis-Hinweis: enthält Audio und Video -> getrennte Weiterverarbeitung möglich.")
elif video_found:
    print("Praxis-Hinweis: Nur Video vorhanden.")
elif audio_found:
    print("Praxis-Hinweis: Nur Audio vorhanden.")
else:
    print("Keine auswertbaren Audio-/Videostreams gefunden.")
```

## Beispiel 16: Videodatei analysieren

Ausgabe:

Datei: trailer\_1080p.ogg  
Container: ogg  
Gesamtdauer: 33.0 s  
Stream 0: video, Codec=theora  
 Auflösung: 1920 x 1080  
 FPS: 25/1  
Stream 1: audio, Codec=vorbis  
 Samplerate: 48000  
 Kanäle: 2  
Praxis-Hinweis: Datei enthält Audio und Video -> getrennte Weiterverarbeitung ist möglich.

Leiten Sie aus der Ausgabe ab, welche Zusatzprüfung vor dem nächsten Arbeitsschritt sinnvoll ist.

## Beispiel 16: Videodatei analysieren

Die Datei ist eine ogg-Containerdatei mit einem theora-codierten Videostream. Das Video hat 1920×1080 Pixel, läuft mit 25 Bildern pro Sekunde, enthält einen Audiostream, der vorbis-Codiert ist. Die Gesamtdauer beträgt 33.0 s.

## Ogg-Format (Container)

### Ogg

- offener, freier Multimedia-Container
- aus dem Xiph-Umfeld
- formal als Ogg Encapsulation Format in RFC 3533 von 2003.
- „nativer Datei- und Stream-Container für Xiph-Codecs“
- „general, freely-available encapsulation format for media streams“

## Theora und Vorbis (Codecs)

### Theora

- freier, offener Video-Codec der Xiph.org Foundation
- nicht mehr ganz modern / klassisch 😊

### Vorbis

- freier, offener, verlustbehafteter Audio-Codec für allgemeine Audiokompression

## Demuxen: aus einer AV-Datei einzelne Streams lösen

Demuxer: Komponenten, die

- Eingaben lesen
- elementare Streams erfassen
- Pakete zur weiteren Verarbeitung ausgeben
- Nutzung:
  - nur Audio für Analyse
  - nur Video für OpenCV
  - Diagnose vor der Weiterverarbeitung



## Beispiel-Code 17: Audio extrahieren

Auswahl des Streams mit `-map 0:a:0`

Audio als WAV exportieren

mit `wave` prüfen:

- Kanäle, Samplerate (Abtastfrequenz), Dauer

Einsatz:

- Interview automatisch verschriftlichen; Pegel und Pausen im Ton untersuchen; Kameraaudio mit externer Aufnahme vergleichen; Audiomerkmale für spätere Klassifikation berechnen

→ Transkription, Audioanalyse, Qualitätsprüfung, Feature-Extraktion

## Beispiel-Code 17: Audio extrahieren

`-map 0:a:0`

Interpretation:

`-map` → Option zur manuellen Streamauswahl

`0` → die erste Eingabedatei

`a` → Audio-Stream

FFmpeg verwendet `:a` als Stream-Specifier für Audio, analog zu `:v` für Video.

`0` → erster Audiostream in dieser Eingabedatei

## Beispiel-Code 17: Audio extrahieren

### Transkription

- Sprachinhalt automatisch verschriftlichen; z. B. Interviews, O-Töne, Vorlesungsaufnahmen
- Vorbereitung für Untertitelung, Suche im Material oder Inhaltserschließung

### Audioanalyse

- Pegelverlauf untersuchen
- Lautstärkeunterschiede oder Pausen erkennen
- Sprache, Musik oder Ereignisse grob unterscheiden
- Spektrogramm berechnen

## Beispiel-Code 17: Audio extrahieren

### Qualitätsprüfung

- prüfen, ob überhaupt Ton vorhanden ist
- sehr leise / übersteuerte / verrauschte Aufnahmen erkennen
- Mono/Stereo, Samplerate und Dauer kontrollieren
- Vergleich zwischen Kameraaudio und externem Audiorecorder

### Feature-Extraktion

- einfache Merkmale aus dem Signal berechnen, z. B.:
  - RMS / Energie
  - Zero-Crossing-Rate
  - spektraler Schwerpunkt
  - MFCCs
  - Onsets / Peak-Zeitpunkte
- Grundlage für Klassifikation oder Segmentierung

## Beispiel-Code 17: Audio extrahieren

```
import subprocess
import wave

INPUT = "trailer_1080p.ogg"
OUTPUT = "audio_for_analysis.wav"
subprocess.run([
    "ffmpeg", "-y",      "-i", INPUT,      "-map", "0:a:0",      "-ac", "1",
    "-ar", "16000",      OUTPUT
], check=True)

with wave.open(OUTPUT, "rb") as wf:
    channels = wf.getnchannels()
    framerate = wf.getframerate()
    nframes = wf.getnframes()
    duration = nframes / framerate

print("Erzeugte Datei:", OUTPUT)
print("Kanäle:", channels)
print("Samplerate:", framerate)
print("Dauer:", round(duration, 2), "s")
```

## Beispiel-Code 17: Audio extrahieren

Ausgabe:

```
Erzeugte Datei: audio_for_analysis.wav
Kanäle: 1
Samplerate: 16000
Dauer: 33.0 s
```

## Aufgabe 10 - Transkription

Nehmen Sie ein kurzes Video mit Sprache auf.

Extrahieren Sie die Audiospur WAV, und lassen Sie sie mit einem Sprach-zu-Text-System transkribieren.

Stellen Sie fest, ob die Audiospur für eine automatische Transkription geeignet ist.

## Aufgabe 11 - Audioanalyse

Laden Sie die extrahiert WAV-Datei in Audacity.

Bestimmen Sie hier

1. Dauer,
2. Pegelverlauf,
3. stille Abschnitte,
4. ein Spektrogramm bestimmt.

Stellen Sie fest:

5. Wo im Clip wird gesprochen?
6. Gibt es längere Pausen?
7. Ist der Sprachsignalpegel gleichmäßig oder stark schwankend?

## Aufgabe 12 - Qualitätsprüfung

Prüfen Sie die extrahiert WAV-Datei dahingehend,

1. ob sie sehr leise ist
2. ob Clipping auftritt
3. ob nur ein Kanal vorhanden ist
4. ob die Dauer zur Dauer der Videodatei passt

Stellen Sie fest:

5. Ist der Ton für weitere Verarbeitung überhaupt geeignet, oder sollte man lieber eine anderes Aufnahmegerät verwenden?

## Muxen: aus getrennten Streams eine Datei erzeugen

Muxer: Komponenten, die

- codierte Pakete
  - empfangen
  - interleaven
  - in die Ausgabedatei schreiben
- Nutzung:
  - neuer Ton zu Video
  - getrennt bearbeitete Streams zusammenführen

## Beispiel-Code 18: AV-Datei demuxen und muxen

Videostream extrahieren

Audiostream extrahieren

beide wieder zusammenführen

-c copy als Streamcopy (ohne Decoding, Filtern, Encoding)

## Beispiel-Code 18: AV-Datei demuxen und muxen

```
import subprocess
def run(cmd):
    print(">>>", " ".join(cmd))
    subprocess.run(cmd, check=True)

input_file = "Sintel_Trailer.480p.DivX_Plus_HD.mkv"

# 1) Videostream herauslösen
run([
    "ffmpeg", "-y", "-i", input_file, "-map", "0:v:0", "-c", "copy", "video_only.mp4"
])

# 2) Audiostream herauslösen
run([
    "ffmpeg", "-y", "-i", input_file, "-map", "0:a:0", "-c", "copy", "audio_only.m4a"
])

# 3) Beide wieder zusammenführen
run([
    "ffmpeg", "-y", "-i", "video_only.mp4", "-i", "audio_only.m4a", "-c", "copy",
    "remuxed_output.mp4"
])
```

## Beispiel-Code 18: AV-Datei demuxen und muxen

Ausgabe:

```
ffmpeg -y -i Sintel_Trailer.480p.DivX_Plus_HD.mkv -map 0:v:0 -c copy video_only.mp4  
ffmpeg -y -i Sintel_Trailer.480p.DivX_Plus_HD.mkv -map 0:a:0 -c copy audio_only.m4a  
ffmpeg -y -i video_only.mp4 -i audio_only.m4a -c copy remuxed_output.mp4
```

## Aufgabe 13 – extrahierte Dateien vergleichen

Vergleichen Sie `input.mp4`, `video_only.mp4`, `audio_only.m4a` und `remuxed_output.mp4` mit `ffprobe` auf Container-, Stream- und Codec-Ebene.

Beschreiben Sie für jede Datei, wofür sie in einem weiteren Arbeitsprozess verwendet werden könnte.

*Dauer: 6 Minuten*

## Aufgabe 13 – extrahierte Dateien vergleichen

`input.mp4`: Ausgangsmaterial zur Inspektion und zum Extrahieren der Streams  
`video_only.mp4`: geeignet für reine Videoanalyse oder Bildverarbeitung  
`audio_only.m4a`: geeignet für Audioanalyse oder Transkription  
`remuxed_output.mp4`: geeignet als zusammengeführte Ausgabedatei zur Wiedergabe oder Weitergabe

## Aufgabe 14 – Extraktion in Arbeitsformat

Mit

```
ffmpeg -i input.mp4 -map 0:a:0 -ac 1 -ar 16000 output.wav input.mp4
```

wird der Audiostream

- extrahiert,
- decodiert, und
- in ein Arbeitsformat umgewandelt: Mono, 16 kHz, WAV/PCM

1. Warum könnten Mono und 16 kHz sinnvoll sein, und
2. was verliert man dabei?

*Dauer: 3 Minuten*



## Framerate und Samplerate: zwei Medien, zwei Zeitraster

### Audio

- Samples
- Samplerate z.B. 48 000 Hz

### Video

- Frames
- Framerate, z.B. 25 fps

→ beide sind zeitbasiert

→ beide sind unterschiedlich diskretisiert

wave dokumentiert Audio-Samplerate und Anzahl der samples, OpenCV dokumentiert FPS für Video

## Framerate und Samplerate: zwei Medien, zwei Zeitraster

### Audio

- Samples
- Samplerate z.B. 48 000 Hz

### Video

- Frames
- Framerate, z.B. 25 fps

→ beide sind zeitbasiert

→ beide sind unterschiedlich diskretisiert

wave dokumentiert Audio-Samplerate und Anzahl der samples, OpenCV dokumentiert FPS für Video



<https://www.blackmagicdesign.com/de/products/davinciresolve>

## Video-Schnitt-Programme

- Kdenlive – offene, leistungsfähige Allround-Lösung für Mehrspur-Schnitt; frei, Open Source und plattformübergreifend. <https://kdenlive.org/de>
- Shotcut – offene, eher technisch orientierte Alternative mit starker Formatunterstützung und nativer Timeline-Bearbeitung. <https://www.shotcut.org/>
- OpenShot – offene, eher einsteigerfreundliche Alternative für einfache bis mittlere Schnittaufgaben. <https://www.openshot.org/>
- DaVinci Resolve – kostenlose, aber nicht Open-Source Profi-Alternative mit sehr großem Funktionsumfang für Schnitt, Farbe und Audio. <https://www.blackmagicdesign.com/products/davinciresolve>

## Zeitbezug: Dauer, Position und Zeitstempel

Zu unterscheiden

- Gesamtdauer
- aktuelle Position
- Zeitbezug eines Samples, eines Frames
- Befehle Video:
  - `CAP_PROP_POS_FRAMES` *Die aktuelle Position im Video als Frameindex.*
  - `CAP_PROP_POS_MSEC` *Die aktuelle Position im Video in Millisekunden.*
  - `CAP_PROP_FPS` *Die Framerate des Videos, also Bilder pro Sekunde.*
- Befehle Audio:
  - `getnframes()` *Die Anzahl der Audioframes in der WAV-Datei.*
  - `getframerate()` *Die Sampling Frequency bzw. Samplerate des Audios.*

## Zeitbezug: Dauer, Position und Zeitstempel

Zu unterscheiden

- Gesamtdauer
- aktuelle Position
- Zeitbezug eines Samples, eines Frames
- Befehle Video:
  - `CAP_PROP_POS_FRAMES` *Die aktuelle Position im Video als Frameindex.*
  - `CAP_PROP_POS_MSEC` *Die aktuelle Position im Video in Millisekunden.*
  - `CAP_PROP_FPS` *Die Framerate des Videos.*
- Befehle Audio:
  - `getnframes()` *Die Anzahl der Audioframes in der WAV-Datei.*
  - `getframerate()` *Die Sampling Frequency bzw. Samplerate des Audios.*

„Audioframe“ eine Zeiteinheit des Audiosignals. Bei Mehrkanalton gehört zu einem Audioframe jeweils ein Sample pro Kanal.

## Zeitbezug: Dauer, Position und Zeitstempel

Zeitliche Zuordnung durch Timestamps (Zeitstempel)

wichtig für

- Wiedergabe
- Muxing, Demuxing
- Schneiden
- Synchronisation

Filter zur Veränderung von Präsentationszeitstempeln

- `setpts`
- `asetpts`

# Zeitstempel

Ein Timestamp (Zeitstempel; Zeit“etikett“) legt für Video-Frame oder Audioblock fest, an welcher Stelle er abgespielt werden soll.

Video: frame-basiert,

Beispiel: Clip mit 25 fps:

- Frame 0 → Zeitposition 0,00 s
- Frame 1 → Zeitposition 0,04 s
- Frame 2 → Zeitposition 0,08 s

Audio: physikalisch: sample-basiert.

Praktisch: in Blöcken (Audioblöcke aus Decoder, Audioframes aus Codes, Fensterbreite eine Analyse, Samples die in einem Puffer enthalten sind.

Beispiel: 480 Samples pro Block:

- Block 0 → 0,00 s
- Block 1 → 0,01 s
- Block 2 → 0,02 s
- Block 3 → 0,03 s

## Aufgabe 15 – Ruckler (Timestamps)

Eine AV-Datei hat die erwartete Gesamtdauer von 33 s. Trotzdem treten beim Abspielen Sprünge, Ruckler oder A/V-Versatz auf. Erläutern Sie, warum die Gesamtdauer allein keine ausreichende Beschreibung des Zeitverhaltens ist.

## Zeitstempel

- `setpts` verändert die Zeitstempel von Videoframes
- `asetpts` verändert die Zeitstempel von Audioframes
- typische Nutzungen:
  - Zeitbezug bei 0 starten lassen
  - Video beschleunigen oder verlangsamen
  - Audio-/Videozeit neu aufsetzen
- Beispiele:
  - `setpts=PTS-STARTPTS` *Videostrom beginnt zeitlich wieder bei null*
  - `setpts=0.5*PTS` *Video wird schneller*
  - `setpts=2.0*PTS` *Video wird langsamer*
  - `asetpts=N/SR/TB` *Audio-Zeitstempel wird aus der Sampleposition neu aufgebaut*

## Latenz: Verzögerung entlang der Pipeline

- Verzögerung, die durch die Verarbeitungskette entsteht
- mögliche Stellen:
  - Aufnahme
  - Buffering
  - Decoding
  - Verarbeitung
  - Encoding
  - Ausgabe

## Aufgabe 16 - Latenz

- a.) Welche Latenzen wären bei einer Datei-Wiedergabe eher unkritisch?
- b.) Welche sind bei Live-Anwendungen problematisch?
- c.) An welchen Stellen der AV-Pipeline kann Latenz entstehen, und warum ist es fachlich wichtig, zwischen konstanter Verzögerung und driftendem Zeitversatz zu unterscheiden?

## Grundlegende Audio-Video-Synchronisation

AV-Sync bedeutet

- zusammengehörige Bild- und Tonereignisse liegen wahrnehmbar passend auf der Zeitachse
- drei Arten von Problemen
  - Startversatz
  - Drift
  - Instabilität, Schwankung
- Diagnosefragen:
  - Starten Audio und Video zueinander passend?
  - Laufen beide über die Dauer wahrnehmbar zusammen?
  - Bleibt der Versatz stabil?

## Aufgabe 17 – Diagnose für Latenzart

Entwerfen Sie einen einfachen Diagnoseplan, mit dem Sie entscheiden können, ob

- a.) ein Startversatz,
- b.) Drift oder
- c.) instabile Synchronität

vorliegt.

## Typische Synchronisationsprobleme in der Praxis

- Lippenbewegung und Ton leicht versetzt
- externer Ton driftet langsam weg
- nach Bearbeitung oder Schnitt wirkt die Datei formal korrekt, aber Bild und Ton passen nicht
- Zeitbezug wurde verändert, ohne den anderen Strom mitzudenken
- Framerate / Samplerate falsch interpretiert

Ordnen Sie drei der genannten Probleme jeweils einer plausiblen Stelle der Pipeline zu.

## Remuxing, Transcoding, Schneiden, Konvertieren

- Remuxing
  - neue Verpackung
  - gleiche codierte Streams
- Transcoding
  - mindestens ein Stream wird neu codiert
- Schneiden
  - zeitlicher Ausschnitt wird erstellt
  - mit oder ohne Re-Encoding

„Konvertieren“: ungenauer Oberbegriff

## Remuxing

- Remuxing = Streamcopy in einen neuen Container.
- Streamcopy:
  - Kopieren von Paketen ohne Decoding, Filterung, Encoding
  - üblicher Befehl: `-c copy`
- schnell
- kein zusätzlicher Qualitätsverlust
- Grenzen:
  - Container-Kompatibilität
  - keine Filterung
  - keine inhaltliche Veränderung

Die ausgewählten Streams werden unverändert kopiert, also nicht neu decodiert und nicht neu encodiert.



## -c copy

- üblicher Befehl: `-c copy`
- schnell
- kein zusätzlicher Qualitätsverlust
- Grenzen:
  - Container-Kompatibilität
  - keine Filterung
  - keine inhaltliche Veränderung

## Transcoding und Schneiden

- Transcoding
  - neuer Codec oder neue Parameter
  - ist langsamer als Remuxing
  - möglicher Qualitätsverlust
- Schneiden:
  - übliche Befehle: `-ss`, `-t`, `-to`
  - Streamcopy ist nicht automatisch frame-genau

## Transcoding und Schneiden

- Transcoding
  - neuer Codec oder neue Parameter
  - ist langsamer als Remuxing
  - möglicher Qualitätsverlust
- Schneiden:
  - übliche Befehle: `-ss`, `-t`, `-to`
  - Streamcopy ist nicht automatisch frame-genau

Nennen Sie zu jedem dieser Fälle einen passenden Begriff:

1. MP4 in MKV ohne Neucodierung
2. Video auf andere Auflösung bringen
3. Clip auf 10 Sekunden kürzen
4. Audio ersetzen und neu ausgeben

## Zusammenfassung

- eine AV-Datei ist ein Container mit elementaren Streams
- vor jeder Weiterverarbeitung muss der Zeitbezug geprüft werden
- Demuxen, Muxen und die Unterscheidung zwischen Remuxing und Transcoding - Arbeitsschritte realer Medienworkflows